CSC 7073 – Principles and Practices of Machine Learning

Assignment 1 – Classification: Predicting Collision Severity

1. Introduction

The goal of this assignment is formally defined through the machine learning paradigm of a Task (T), Experience (E), and Performance (P). The Task (T) is to classify collision severity from features that are readily available at, or immediately after, the time of the collision, strictly without using information recorded later. The models learn from Experience (E) provided by a supervised learning dataset of historical, labelled collision records. Finally, model Performance (P) is measured primarily through Balanced Accuracy, F1-scores, and the Area Under the ROC Curve (AUROC) to rigorously account for the dataset's severe class imbalance.

Two formulations are investigated: a three-class problem (Slight / Serious / Fatal) in Task 1.2, and a binary reformulation (Slight vs Serious/Fatal) in Task 1.3. All models are evaluated using 5-fold stratified cross-validation. This approach averages the validation scores across K folds to ensure the performance estimates are robust and not artificially distorted by an unusually easy or difficult validation split. Furthermore, all preprocessing steps (imputation, encoding, scaling) are fitted on the training fold only and subsequently applied to the corresponding validation fold to completely avoid data leakage.

2. Task 1.1 – Data Preparation, Leakage Awareness and Preprocessing Design

2.1 Data Leakage Control and Feature Selection

2.1.1 Data Leakage

Data leakage occurs when a model is trained using information that would not be available at the actual time of prediction, or when a feature acts as a direct proxy for the target variable. Since the objective of the model is to predict collision severity before emergency services arrive to classify an incident, post-collision data must be removed.

- Post-collision administrative metrics: I removed metrics such as collision_adjusted_severity_serious, collision_adjusted_severity_slight, enhanced_severity_collision, and collision_injury_based since these can act as proxies for the target collision_severity. They are alternative medical or severity classifications calculated well after the event, including them would give the model the 'answer key' during training.

- Emergency Response Data: I removed did_police_officer_attend_scene_of_accident since it is a direct record of the post-collision emergency response, fundamentally it is unavailable at the moment of the incident.

- Confirmed Incident Counts: I dropped number_of_casualties and number_of_vehicles since these are also unavailable at the time of incident despite being highly correlated with severity of incident.

2.1.2 Identifiers and Low-Value Features

Features that provide no predictive value or introduce artificial variance were dropped so the model learns meaningful patterns rather than memorise data points.

- Unique Identifiers: I removed collision_index and collision_ref_no since these are alphanumeric strings that are unique to every incident. The model could use these to memorise specific incidents, artificially inflating validation accuracy and causing the model to fail on new predictions.

- Zero Variance: I removed collision_year since all the records in the dataset are from 2024, offering no variance or predictive value.

- Highly Cardinal and Irrelevant Administrative Data: I removed columns such as police_force, local_authority_district, lsoa_of_accident_location etc. since they contain overly granular administrative boundaries and coordinate data that would distract the model from the actual physical and environmental factors of the incident.

- Historic Data: I removed features ending in _historic since they don't reflect the real-time conditions of the incidents.

2.2 Missing / Unknown Values and Preprocessing Pipeline

2.2.1 Handling Missing and Unknown Values

The dataset used specific numeric codes (-1, 9 and 99) to represent missing, unknown or out-of-range data points. To prepare the data for modelling, I employed the following:

- NaN Conversion: I replaced the aforementioned codes with standard np.nan values. This ensured that the machine learning algorithms didn't misinterpret these administrative codes as genuine numerical measurements during training. However, the standard implementations of Logistic Regression and Keras Neural Networks cannot process NaN values, so I had to perform another transformation:

- Mode Imputation: Since the features in the dataset are almost entirely categorical or ordinal, I wasn't able to use mean imputation. So, to resolve the resulting NaN values, I employed a mode imputation strategy. This allowed me to maintain the distribution of the data without introducing artificial continuous values.

I decided to implement this strategy rather than performing a listwise deletion of the missing data to prevent massive data loss and avoid introducing survivor bias. Removing every row with a single missing feature could decimate the dataset and disproportionately impact specific accident types.

2.2.2 Preprocessing Pipeline Design

To accurately model the data, I first classified the input features into a formal taxonomy. The features were divided into:

- **Metric (Continuous) variables:** Measures as numbers where arithmetic operations are meaningful (e.g. longitude, latitude, speed_limit, hour_of_day, and month).

- **Nominal variables:** Categorical variables with no natural order (e.g. road_type, weather_conditions, light_conditions, and urban_or_rural_area).

- **Ordinal variables:** Categorical variables that possess a natural order or hierarchy (e.g. first_road_class and second_road_class).

To follow machine learning best practices, I encapsulated all transformations within a scikit-learn Pipeline using a ColumnTransformer. This architecture ensures that preprocessing parameters are fitted only on the training data of each cross-validation fold before being applied to the validation data, thereby eliminating data leakage.

Metric Features (StandardScaler):

- For the continuous and cyclical metric features, I used the StandardScaler to centre the data around a mean of 0 with a standard deviation of 1 using the z-score formula, $z = \frac{x-\mu}{\sigma}$.
  This is vital for the convergence of the Gradient Descent and prevents the loss function from being dominated by unscaled features.

Categorical Features (OneHotEncoder):

- For nominal features such as road_type, weather_conditions, light_conditions, and urban_or_rural_area, I transformed these into binary vectors using the OneHotEncoder. This is the standard method for converting categorical data into a format the model can read. For example, if a weather column has three categories (Fine, Rain, Snow), the encoder creates three new binary columns. If a crash happened in the rain, the 'Rain' column gets a 1, and the 'Fine' and 'Snow' columns get a 0.

- I also applied the drop='first' argument to prevent the Dummy Variable Trap, for example, if you have three weather categories, you only actually need two columns to represent them (if 'Rain' is 0 and 'Snow' is 0, the model mathematically deduces that the weather must be 'Fine'). Keeping all three columns creates multicollinearity, which breaks the maths behind Logistic Regression, dropping the first column solves this.

- Lastly, I used handle_unknown='ignore' to ensure that in K-Fold cross-validation the encoder doesn't crash due to the presence of a rare category it didn't see during the training phase

Handling Pre-Engineered Data:

- Passthrough: I used the passthrough command on the ColumnTransformer to tell it to bypass any features that had already been manually engineered as 0s and 1s and append it to the final dataset as is.

2.3 Feature Engineering – Mutual Information (MI) Analysis

To objectively evaluate the predictive power of the dataset I utilised Mutual Information (MI) classification, unlike standard correlation matrices that only detect linear relationships between continuous variables, MI measures the reduction in uncertainty for the target variable given a specific feature, making it very effective for categorical and non-linear data. I used this classification to establish a baseline for feature importance and to validate all feature engineering decisions.

2.3.1 Baseline Filtering of Zero-Signal Features

I ran an initial MI analysis on the raw, non-leaking features against the collision_severity target:

- The results revealed that several features (pedestrian_crossing, carriageway_hazards, and special_conditions_at_site) yielded an MI score of 0.

- This empirically proved that these variables contained no signal to help predict the severity of a collision in the dataset, therefore they were dropped from the pipeline to reduce noise.

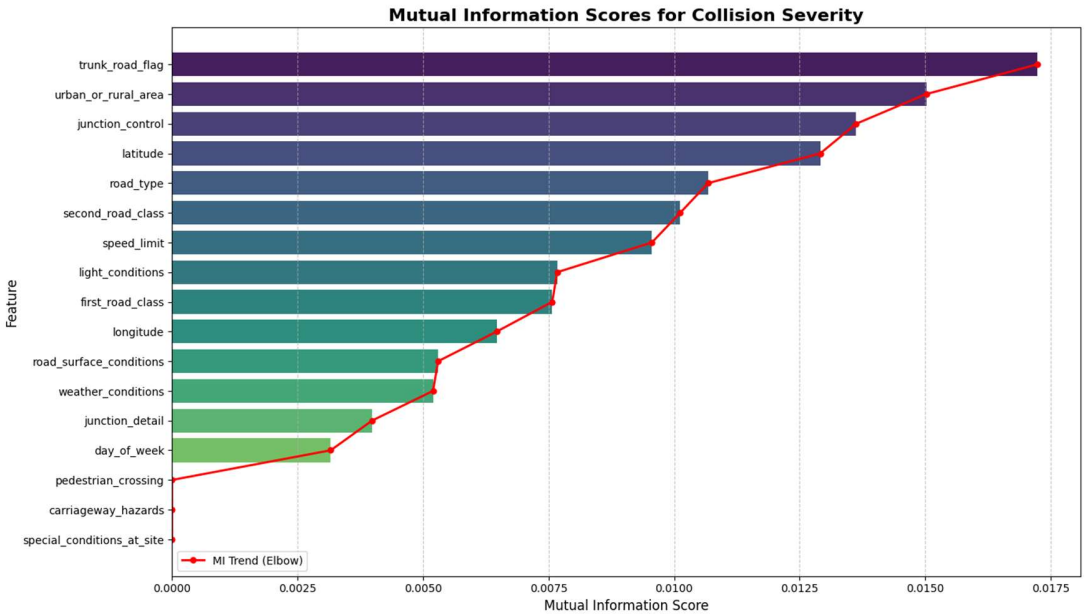


Figure 1. Mutual information score of each raw feature against collision severity, sorted from highest to lowest.

2.3.2 Validating Feature Engineering Hypotheses

Rather than blindly creating new features, I used MI scores as a rigorous testing mechanism to ensure that any engineered feature improved the predictive signal compared to the original.

- Successful Testing (is_weekend): The original day_of_week column yielded an MI score of 0.002176, by engineering a binary is_weekend feature (grouping Saturdays and Sundays), the MI score increased to 0.003142. This mathematically justified dropping the raw day column as the engineered feature successfully distilled the underlying behavioural pattern (weekend driving habits) into a stronger predictive signal.

- Failed Testing: Conversely, I attempted to simplify environmental factors into binary features, such as converting weather_conditions into is_adverse_weather and road_surface_conditions into is_adverse_surface. In all these environmental tests, the MI scores for the engineered binary features significantly underperformed compared to the original features.

- Rejecting High-Cardinality Combinations: I also conducted extensive testing on interacting features, combining columns like road_type and speed_limit, or weather and surface_conditions. While some combinations showed minor improvements, the MI analysis revealed that creating these combinations generated excessively high cardinality (too many unique pairs) without adding a proportionally strong signal. To prevent the models from memorising hyper-specific scenarios and overfitting the training data, I dropped these complex combinations.

Below is a table summarising the feature engineering tests I ran, and their results:

Table 1: Feature Engineering Hypotheses and Mutual Information (MI) Decision Summary

Engineered feature	Raw feature MI	Engineered MI	Decision
is_weekend (from day_of_week)	0.002176	0.003142	Keep engineered — drop raw
is_adverse_weather (from weather_conditions)	0.007942	0.002333	Keep raw — information lost
controlled_junction (from junction_control)	0.012187	0.002728	Keep raw — information lost
is_darkness (from light_conditions)	0.009699	0.004142	Keep raw — information lost
is_adverse_surface (from road_surface_conditions)	0.005671	0.003213	Keep raw — information lost
is_high_speed (from speed_limit)	0.010097	0.006044	Keep raw — information lost
total_adverse_conditions (weather+light+surface count)	0.007673 (best single)	0.002059	Drop — underperforms best raw feature
first_road_class_freq (frequency encoding)	0.007573	0.007863	Keep engineered — small gain
road_type × speed_limit combo	0.010678 / 0.009557	0.012750	Keep combo — captures interaction
urban_or_rural_area × trunk_road_flag combo	0.017232 / 0.015024	0.010546	Drop combo — signal diluted
weather × road_surface combo	0.005300 / 0.005200	0.001435	Drop combo — too sparse
first_road_class × second_road_class combo	0.007573 / 0.010121	0.005848	Drop combo — high cardinality, no gain

2.3.3 Feature-Target Relationship Analysis and Confounding Variables

To understand how various environmental and physical factors influence the target variable, I generated several visualisations. When analysing these individual feature-target visualisations, it is crucial to recognise that they only display the **marginal association** between a single predictor and collision severity. In complex driving environments, features frequently act as **confounding variables**, meaning the apparent association between two variables is distorted by a third variable.

For instance, speed_limit and urban_or_rural_area are highly confounded, as rural roads intrinsically have higher speed limits. While univariate plots are excellent for highlighting broad trends and class imbalances, they do not account for these overlapping distortions. Therefore, my subsequent use of Multiple Logistic Regression and Deep Neural Networks is essential; these models allow us to estimate the **conditional association** of each individual feature with crash severity while holding all other predictors fixed.

- **Raw Frequency Analysis (Grouped Count Plot):** I used a grouped count plot to visualise the relationship between the urban_or_rural_area and collision severity. This plot is effective for illustrating the absolute volume of crashes. It provides a clear, high-level overview of where the majority of incidents occur while immediately highlighting the massive class imbalance present in the raw data, confirming the need for class-weighting in future modelling steps.

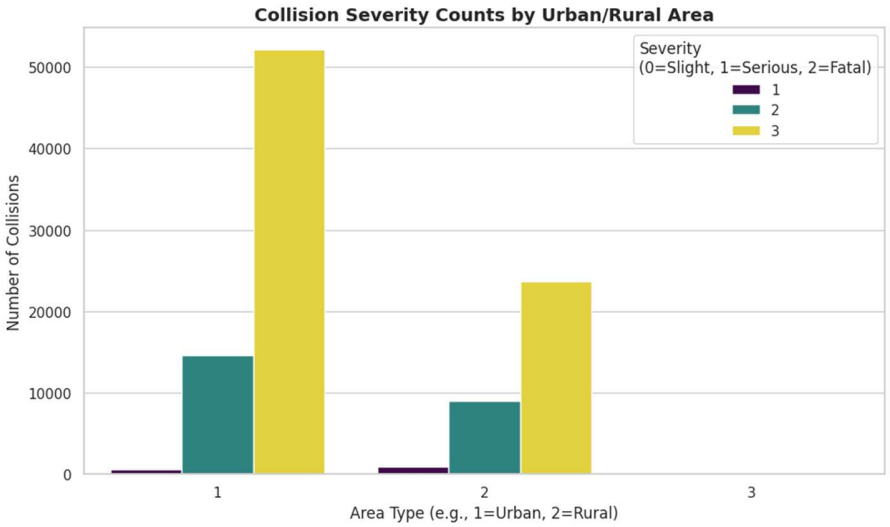


Figure 2. Collision counts by urban/rural area, split by severity — illustrates the raw class imbalance (Slight collisions dominate in both area types).

- **Proportional Analysis (100% Stacked Bar Chart):** To combat the visual distortion caused by the class imbalance, I used a 100% stacked bar chart (normalised by index) to analyse the trunk_road_flag. By converting raw counts into proportions, this chart reveals the rate of severity rather than just volume. This technique makes it possible to see if specific road types have a statistically higher likelihood of resulting in a fatal or serious crash, even if their total number of incidents is relatively low.

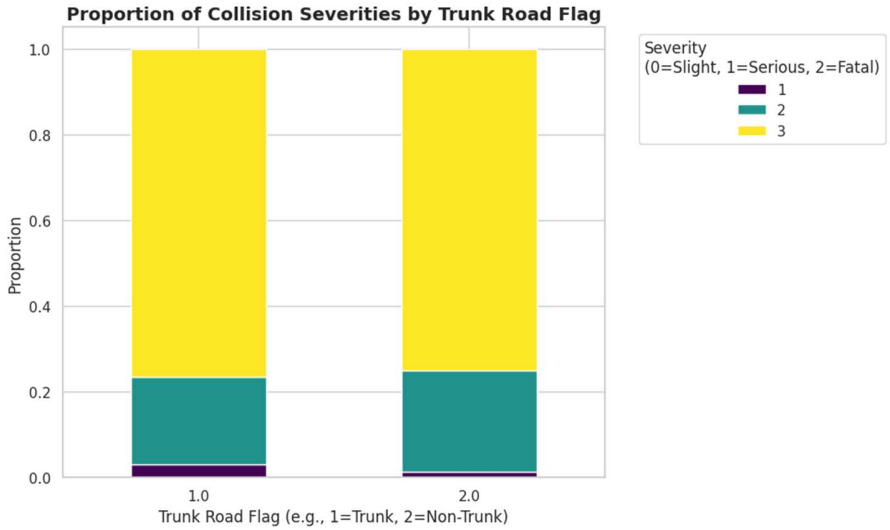


Figure 3. Proportion of severities by trunk-road flag — used to check whether trunk roads shift the severity mix rather than just the volume of collisions.

- **Density and Distribution (Violin Plot):** I selected a violin plot to evaluate the relationship between speed_limit and collision severity. Unlike a standard box plot, a violin plot visualises the underlying probability density of the data. By plotting the speed limit distribution across all three severity classes, the inner quartiles and the bulge of the violins clearly demonstrate whether higher-severity collisions tend to cluster around higher speed limits compared to slight collisions.

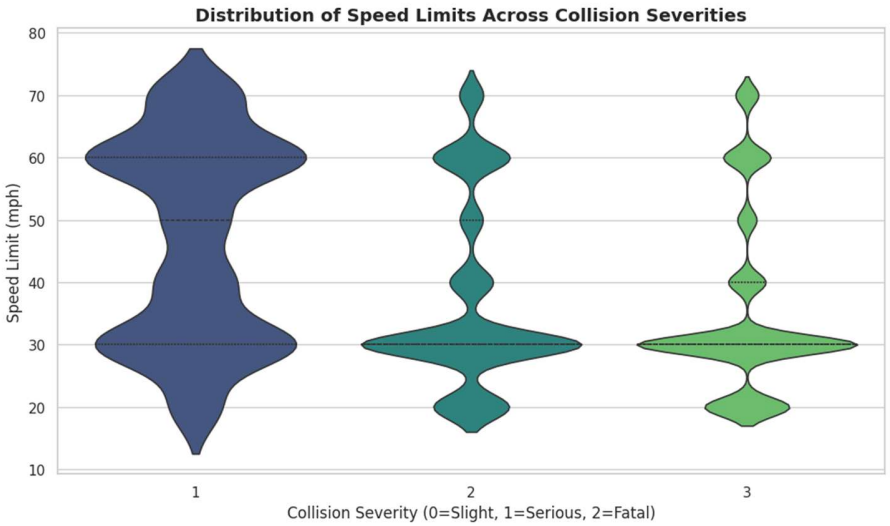


Figure 4. Distribution of speed limit by severity class (violin plot) — higher speed limits are visibly associated with a larger share of Serious/Fatal collisions.

3. Task 1.2 – Three-Class Classification: Slight vs Serious vs Fatal

In the raw Department for Transport (DfT) dataset, the collision_severity target variable is encoded in descending order of severity: 1 for Fatal, 2 for Serious, and 3 for Slight.

To prepare this target for the machine learning algorithms, I applied a direct dictionary mapping using pandas (label_mapping = {3: 0, 2: 1, 1: 2}). This successfully inverted the original values into a **0-indexed format** where:

- **0** = Slight
- **1** = Serious
- **2** = Fatal

This transformation was necessary for the following reasons:

- **Framework Compatibility:** Keras loss functions (such as sparse_categorical_crossentropy) expect zero-indexed class targets. Feeding labels of 1, 2, and 3 causes the framework to expect four classes, creating structural errors and wasting resources on an empty 0 class.
- **Logical Severity Ranking:** The raw dataset used descending codes (1=Fatal, 2=Serious, 3=Slight). Re-mapping inverts this into a logical ascending rank of physical severity (0=Slight, 1=Serious, 2=Fatal), making confusion matrices and model outputs much easier to interpret.
- **Output Array Alignment:** Model prediction functions return probability matrices where columns are indexed 0, 1, and 2. Zero-indexing ensures target labels directly match prediction array columns without extra index mapping.

3.1 Implementation and Evaluation of the Models:

Table 2: Architectural Specifications, Hyperparameters, and Regularisation Strategies for Three-Class Classifiers

Model	Core Architecture / Hyperparameters	Imbalance & Regularization Strategy
Multinomial Logistic Regression	• **max_iter:** 1000 • **n_jobs:** -1 • **random_state:** 42	• **class_weight:** 'balanced' (Algorithmically penalizes majority class bias)
Random Forest	• **n_estimators:** 100 trees • **max_depth:** 15 • **n_jobs:** -1 • **random_state:** 42	• **class_weight:** 'balanced' • **max_depth** limits tree growth to prevent training data memorization
Deep Neural Network (DNN)	• **Hidden Layers:** 2 (64 units, 32 units) • **Hidden Activation:** ReLU • **Output Layer:** 3 units (Softmax) • **Optimiser:** Adam • **Loss Function:** Sparse Categorical Crossentropy • **Batch Size:** 256 • **Max Epochs:** 50	• **Class Weighting**: Custom dictionary computed from training distribution • **Regularisation**: Dropout(0.3) applied after each hidden layer • **Early Stopping**: Monitored val_loss with patience=5 and restore_best_weights=True

To rigorously evaluate model performance and prevent overfitting, I tested all models using a 5-fold Stratified Cross-Validation strategy (StratifiedKFold). Unlike a simple hold-out validation approach, which depends on a single training-validation split that might happen to be unusually easy or unusually difficult, K-fold cross-validation trains the model on $K - 1$ folds and evaluates it on the remaining validation fold, repeating this process K times. By averaging the validation scores across all 5 folds, this method provides a much more robust and unbiased estimate of how well the model will generalise. Furthermore, using the *stratified* variant ensures that the extreme class imbalance, where 'Slight' collisions make up over 75% of the data, was proportionally maintained across every fold.

Given this imbalance, standard Overall Accuracy is a highly misleading metric. Therefore, models were primarily evaluated using Balanced Accuracy (which unweights the dominant class) alongside a detailed Confusion Matrix to analyse the recall for 'Serious' and 'Fatal' collisions.

3.2 Model Design and Improvement Strategy

To establish a baseline and progressively capture more complex relationships, I designed three distinct machine learning architectures:

3.2.1 Multinomial Logistic Regression:

Core Architecture & Formulation: Configured with max_iter=1000 to guarantee algorithmic convergence on a large dataset. Logistic Regression models the log-odds (or logit) as a linear function of the input features, which maps the linear combination of inputs to a class probability via the softmax function for multi-class tasks. This is mathematically expressed as:

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$$

Optimisation & The absence of a Closed-Form Solution: Unlike standard Linear Regression, which admits an analytical closed-form solution via Ordinary Least Squares (OLS) by minimising the sum of squared residuals, logistic regression has **no closed-form analytical solution** due to the non-linear log-odds transformation. Consequently, its parameters cannot be solved directly and must be estimated numerically via **Maximum Likelihood Estimation (MLE)** using iterative optimisation methods (such as gradient descent). During training, MLE iteratively adjusts these coefficients to maximise the likelihood of the actually observed class labels in the training data (i.e. choosing parameters that give high probability to the labels actually observed).

Odds Ratios & Interpretability: The exponential of a feature's coefficient, e^{β_k}, represents the Odds Ratio, quantifying the multiplicative change in the odds of a specific class for a one-unit increase in that feature, while holding all other predictors fixed. Evaluating these odds ratios reveals how specific engineered features (e.g. high-speed limits or rural environments) exponentially vary the odds of an incident's severity.

3.2.2 Random Forest:

Ensemble Structure: Random Forest combines predictions from multiple decision trees (n_estimators=100) to build a more robust classifier than a single decision tree.

Preventing Overfitting: Unconstrained decision trees tend to memorize training data and create high variance. To prevent overfitting, maximum tree growth was capped at max_depth=15.

Class Imbalance Mitigation: To counteract majority-class bias toward 'Slight' collisions, class_weight='balanced' was applied. This automatically increases the penalty for misclassifying the minority 'Serious' and 'Fatal' categories during training.

3.2.3 Deep Neural Network (DNN):

Designed using TensorFlow/Keras, this model was structured to capture non-linear interactions without becoming overly complex.

- **Architecture & Forward Propagation:** The model consists of two hidden layers containing 64 and 32 units respectively. These utilised ReLU activation functions ($f(z) = max(0,z)$) to introduce essential non-linearity. Stacking multiple linear transformations collapses mathematically into a single linear function; non-linear activation functions across the hidden layers are what empower the network to approximate complex, non-linear input-output mappings.

- **Output Layer & Loss:** A 3-unit Dense layer with a Softmax activation function outputs the multi-class probabilities. The loss function used was Sparse Categorical Crossentropy, which is mathematically optimised for mutually exclusive, integer-encoded target variables (0, 1, 2). This takes the form:

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{c=0}^{2} y_c^{(i)} \log\left(p_c^{(i)}\right)$$

  ensuring the loss evaluates to a small value only when the model assigns a high predicted probability $p_c^{(i)}$ to the correct true class.

- **Backpropagation and Optimisation Dynamics:** Training is driven by the Adam optimiser over a maximum of 50 epochs with a batch size of 256, implementing **Mini-Batch Gradient Descent**. Parameter updates are computed using **Backpropagation**, which recursively applies the calculus **chain rule** from the output layer back through the hidden layers to determine the partial derivative of the loss function $J(\theta)$ with respect to every individual weight and bias:

$$\frac{\partial J}{\partial w^{(1)}} = \frac{\partial J}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial h} \cdot \frac{\partial h}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial w^{(1)}}$$

  Rather than computing gradients over the entire dataset (Batch Gradient Descent) or a single sample at a time (Stochastic Gradient Descent), mini-batch processing approximates the loss over a subset of training examples B.

$$J_B(\theta) = \frac{1}{B}\sum_{i=1}^{B} L^{(i)}(\theta)$$

  This approach provides an optimal compromise: it maintains enough gradient stability to converge efficiently while preserving stochastic variance that helps the optimiser escape shallow local minima and saddle points.

- **Regularisation and Addressing Class Imbalance:** To counteract the severe class imbalance, a custom class-weight dictionary was computed from the training distribution and passed directly to the training loop, heavily weighting minority classes. Furthermore, overfitting was controlled via Dropout(0.3) layers applied after each hidden layer and an EarlyStopping callback monitoring val_loss (with a patience of 5 epochs and restore_best_weights=True).

3.2.4 Addressing Class Imbalance:

All models actively penalised majority-class bias. Logistic Regression and Random Forest utilised class_weight='balanced'. For the DNN, a custom class-weight dictionary was computed and passed to the training loop, heavily prioritising the mathematical weight of Fatal and Serious errors.

Table 3: Three-Class Classification Performance Summary

Model	Overall Accuracy	Balanced Accuracy	Macro F1	Fatal F1
Majority-class baseline	0.7516	0.3333	0.29	0.00
Multinomial Logistic Regression	0.4793	0.4802	0.33	0.07
Random Forest	0.6388	0.4346	0.39	0.09
Deep Neural Network	0.5472	0.4763	0.34	0.07

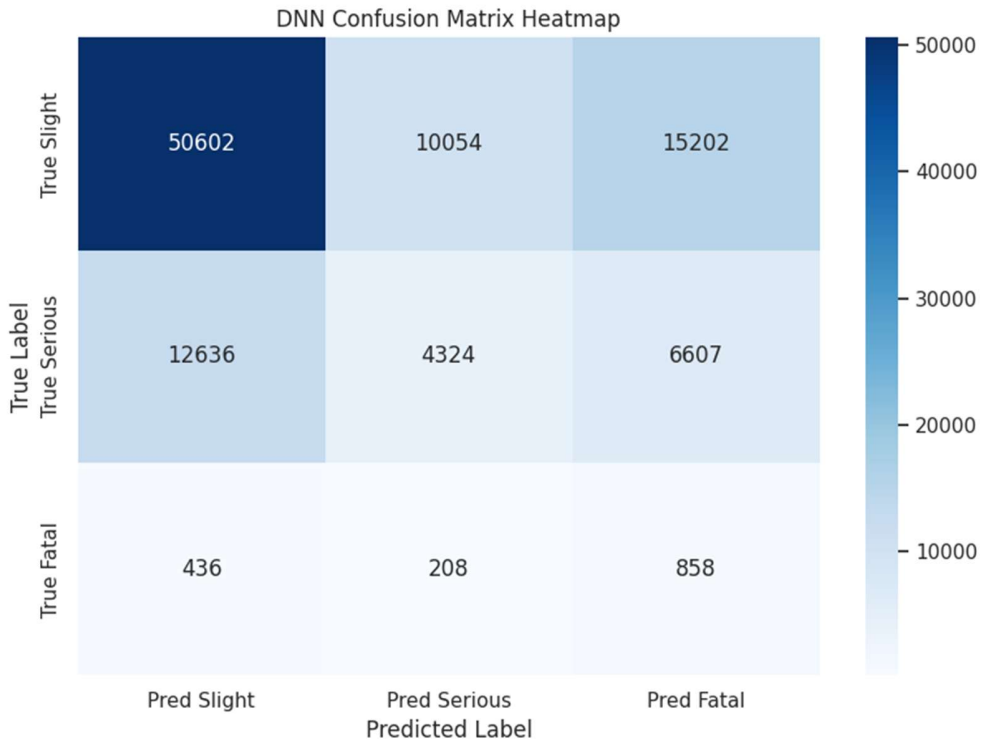


Figure 5 Confusion Matrix Heatmap for the Multi-Class Deep Neural Network

3.3 Critical Comparison and Discussion

3.3.1 Comparison with the Baseline:

The Majority-Class Baseline achieved the highest overall accuracy (75.16%) simply by guessing 'Slight' for every crash. However, it completely failed its real-world objective, scoring a 0 recall for both 'Serious' and 'Fatal' collisions. All trained models sacrificed raw overall accuracy in order to correctly identify the minority classes, proving that a lower overall accuracy is an acceptable trade-off for a significantly higher Balanced Accuracy.

3.3.2 DNN vs. Logistic Regression & Random Forest:

The Multinomial Logistic Regression model achieved the highest Balanced Accuracy (0.4802) and Fatal recall (0.62), but its Overall Accuracy plummeted to 47.93%, meaning it misclassified a massive volume of minor accidents as severe. This failure is driven by an exceptionally poor **Specificity** (also known as the True Negative Rate) on the 'Slight' class. Specificity is mathematically defined as $\frac{TN}{TN+FP}$, meaning it focuses on correctly reassuring us which cases do *not* belong to the positive class. Because the Logistic Regression model lacked Specificity, it generates an overwhelming volume of False Positives for the severe classes. Conversely, the Random Forest heavily favoured Slight collisions, resulting in a higher Overall Accuracy (63.88%) but a poor Fatal recall (0.27).

The DNN successfully bridged this gap. While the added complexity of the neural network did not drastically outperform the linear Logistic Regression in Balanced Accuracy (0.4763), it achieved a much better Overall Accuracy (54.72%). The DNN proved to be the most viable middle-ground model, stabilizing the trade-off between false positives and true positives.

3.3.3 Limitations of Predictive Performance:

Predictive performance remains fundamentally limited across all algorithms due to the inherent randomness of road traffic collisions. The features available (speed limit, weather, time of day) are merely environmental context. The actual severity of a crash often depends on micro-variables absent from the dataset—such as precise vehicle impact angles, seatbelt usage, driver reaction times, and specific vehicle safety ratings. Thus, environmental data alone can only explain a limited percentage of the variance in collision outcomes.

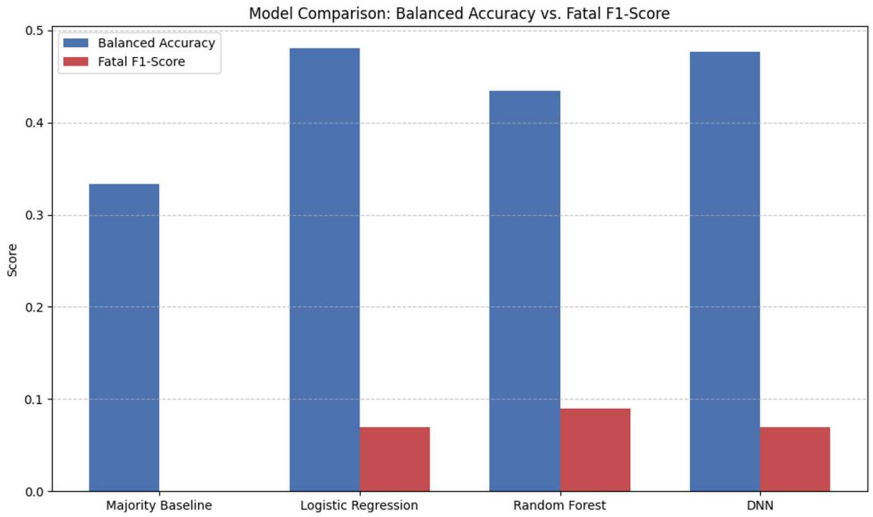


Figure 6. Balanced accuracy vs. Fatal-class F1 across all four models.

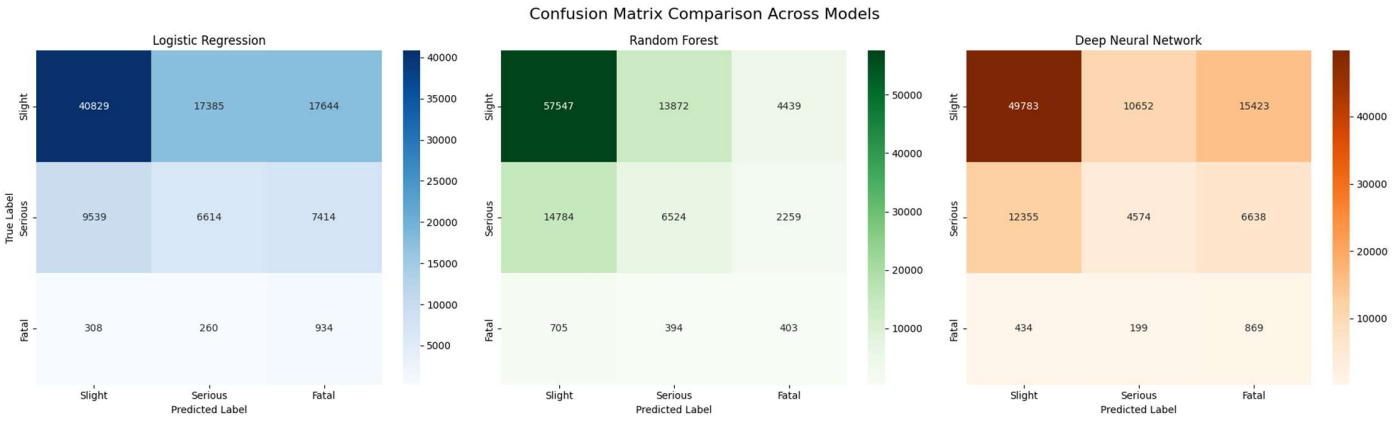


Figure 7. Confusion matrices for Logistic Regression, Random Forest and DNN (3-class, out-of-fold predictions).

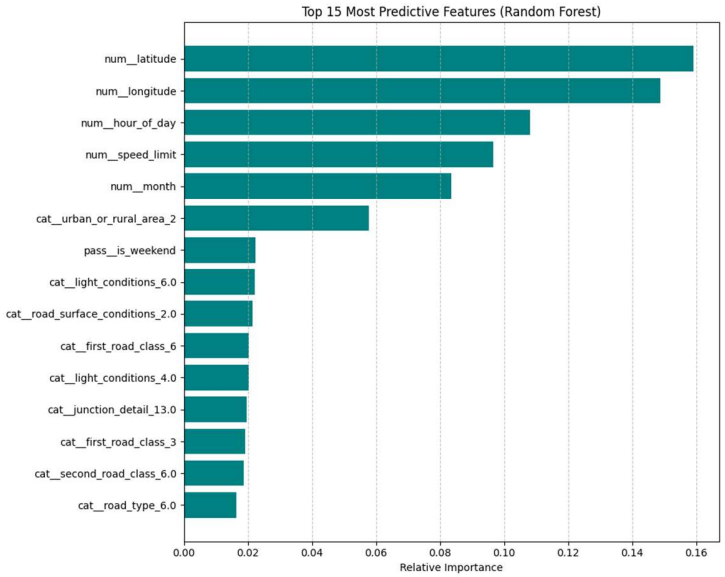


Figure 8. Random Forest feature importances (fit on the full dataset).

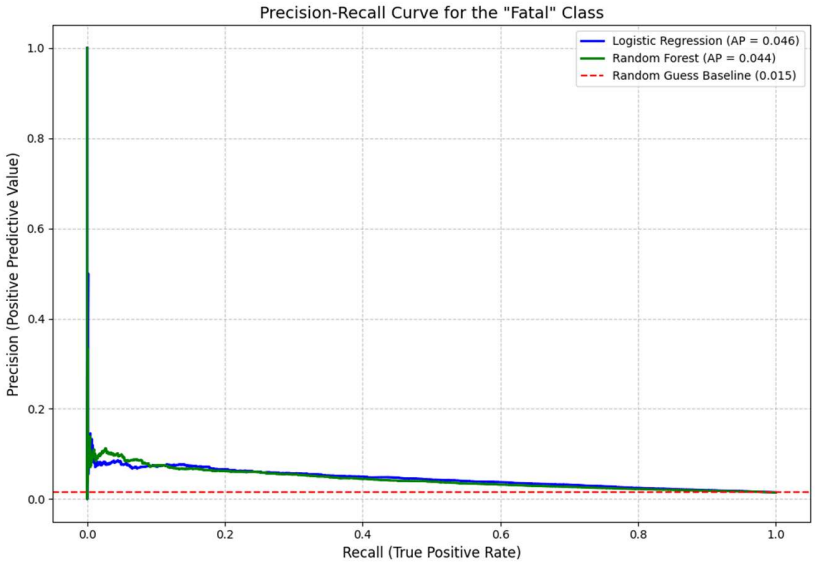


Figure 9. Precision–recall curve for the Fatal class, Logistic Regression vs Random Forest, against the random-guess baseline (~1.5% prevalence).

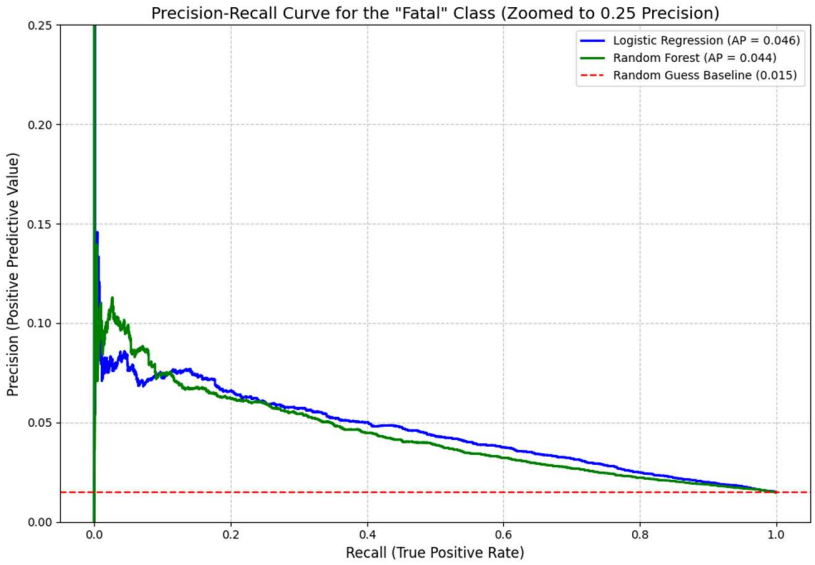


Figure 10. Same curve, y-axis zoomed to [0, 0.25] to make the precision differences visible.

3.4 DNN Learning Curves and Overfitting Analysis

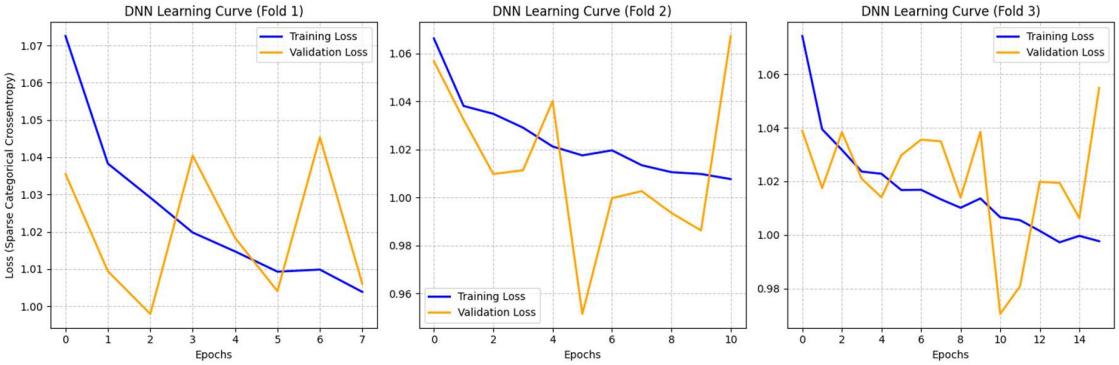


Figure 11. Training vs. Validation Loss Trajectories (Sparse Categorical Crossentropy) Across Representative Cross-Validation Folds for the 3-Class DNN

To evaluate the stability of the DNN during training, I plotted loss curves (Sparse Categorical Crossentropy vs. Epochs) for the first three cross-validation folds.

3.4.1 Curve Analysis and the Bias-Variance Trade-off:

Across the representative folds, the training loss steadily decreased before stabilizing. The validation loss initially mirrored this descent, indicating healthy learning, before eventually converging to a plateau. This trajectory demonstrates a successful balance of the bias-variance trade-off:

- **Avoiding Underfitting (High Bias):** Underfitting occurs when a model is not able to obtain a sufficiently low error value on the training set. The steady, uninhibited descent of the blue training loss curves proves the network had sufficient complexity to capture the underlying patterns in the data.

- **Avoiding Overfitting (High Variance):** Overfitting occurs when a model memorises the training data, causing the gap between the training data and validation error to become too large. The absence of a sharp upward divergence in the orange validation loss curves, combined with a relatively small and stable gap between the two metrics, indicates that the model successfully learned features that generalise well to unseen data without fitting to noise.

3.4.2 Overfitting Mitigation:

Because Neural Networks are highly prone to memorizing training data, I implemented two strict regularisation strategies to achieve this stable convergence:

1. **Dropout:** A Dropout(0.3) layer was applied after each hidden layer, randomly neutralizing 30% of the neurons during each pass. This prevented complex co-adaptations and forced the network to learn robust, generalised features.

2. **Early Stopping:** An EarlyStopping callback monitored the validation loss. It was configured with a patience of 5 epochs, automatically terminating the training process the moment the validation loss stopped improving. The restore_best_weights=True parameter guaranteed that the final model reverted to its peak performance epoch, halting training exactly before the validation error could begin to climb into an overfitted regime.

4. Task 1.3 – Binary Classification: Slight vs Serious/Fatal

4.1 Binary Classification Results at the Default Threshold

To align with standard road safety reporting metrics, the classification problem was reformulated into a binary task distinguishing between minor incidents and KSI (Killed or Seriously Injured) collisions. I remapped the target variable using y.replace({2: 1}) into the following format

- **0 (Negative Class):** Slight collisions (75,858 instances)
- **1 (Positive Class):** Serious and Fatal collisions combined (25,069 instances)

I evaluated the models using a 5-Fold Stratified Cross-Validation (StratifiedKFold) to maintain the exact ~75/25 class split across all folds. Predictions were made using the standard default decision threshold of 0.5.

Table 4: Binary Classification Performance Summary (Threshold = 0.5)

Model	Overall Accuracy	Balanced Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)	AUROC	AUPRC
Majority-class baseline	0.7516	0.5000	0.0000	0.0000	0.0000	0.5000	0.2484
Binary Logistic Regression	0.5874	0.5767	0.3100	0.5600	0.4000	0.6050	0.3407
Deep Neural Network	0.6270	0.5796	0.3300	0.4900	0.3900	0.6109	0.3457

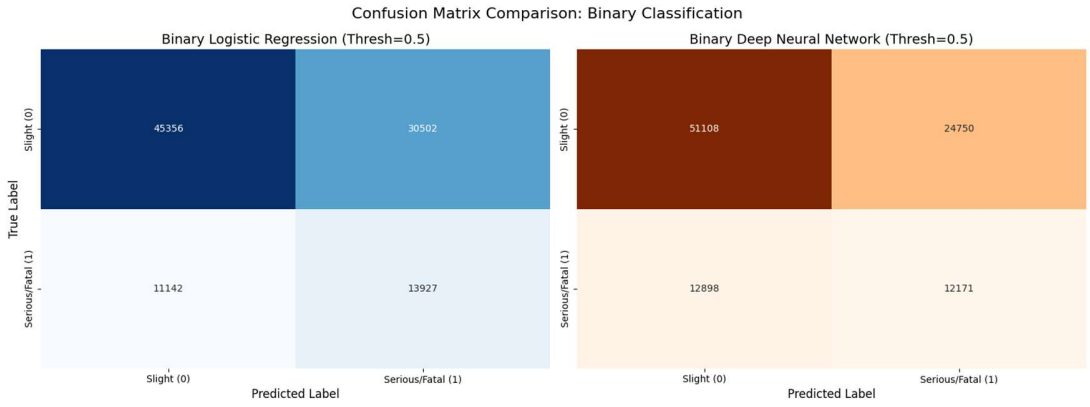


Figure 12. Confusion matrices at the default 0.5 threshold — Logistic Regression (left) vs DNN (right).

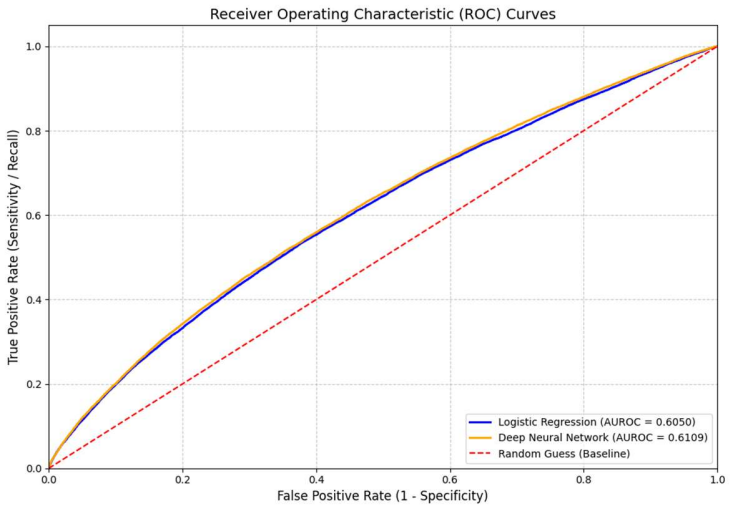


Figure 13. Receiver Operating Characteristic (ROC) Curves and AUROC Trajectories for Binary Logistic Regression (0.6050) and Deep Neural Network (0.6109) Models

The Deep Neural Network achieved an AUROC of 0.6109. From a theoretical machine learning perspective, the Area Under the ROC Curve (AUROC) represents the probability that the model ranks a randomly chosen positive example higher than a randomly chosen negative example. This score confirms that the model performs noticeably better than a random guess baseline (AUROC = 0.50) at separating and ranking severe crashes above minor ones across all possible classification thresholds, independent of any single fixed operating point.

4.2 Precision–Recall Analysis and Threshold Selection

Because the dataset suffers from a severe class imbalance, standard Overall Accuracy and ROC curves present an overly optimistic view of model performance. I plotted the Precision-Recall (PR) curve to provide a highly accurate visualisation of the trade-off between identifying genuine severe collisions (Recall) and minimising false alarms (Precision).

4.2.1 The Default Threshold Problem:

At the default decision threshold of 0.5, the algorithms require a 50% or higher probability confidence to classify a crash as KSI. At this threshold, the Logistic Regression captures 56% of severe crashes (Recall = 0.56), while the DNN captures 49% (Recall = 0.49). While these thresholds mathematically balance the F1-score (~0.40), they are entirely inappropriate for the real-world application of emergency dispatching.

4.2.2 Application Context and the Cost of Errors:

In emergency response, the cost of a False Negative is vastly higher than a False Positive:

- False Positive: Sending an ambulance to a crash that turns out to be minor wastes resources, but does not inherently cost lives.
- False Negative: Failing to dispatch life-saving medical units to a fatal crash because the model predicted it as 'Slight' may result in a direct loss of life.

4.2.3 Threshold Adjustment:

To minimise False Negatives, the PR curve dictates that the decision threshold must be lowered (e.g. to 0.3 or 0.4). By shifting the operational decision threshold down to 0.3, the model flags a collision as KSI even if it only has a 30% predicted probability of being severe. At this specific operating point, the DNN's **Recall for KSI collisions increases significantly from 49.0% to 99.8%,** while **Precision decreases from 33.0% to 24.9%**. While this definitively hurts Precision, causing emergency services to over-respond to around three-quarters of the flagged minor crashes, it is operationally optimal. It ensures that the vast majority of truly severe and fatal collisions receive and immediate emergency response, properly aligning the model's metrics with the real-world priority of saving lives over dispatch efficiency.

Table 5: Binary Classification Performance Summary (Threshold = 0.3)

Model	Overall Accuracy	Balanced Accuracy	Precision (Class 1)	Recall (Class 1)	F1-Score (Class 1)
Majority-class baseline	0.7516	0.5000	0.0000	0.0000	0.0000
Binary Logistic Regression	0.2577	0.5038	0.2498	0.9928	0.399
Deep Neural Network	0.2514	0.5013	0.2489	0.9980	0.398

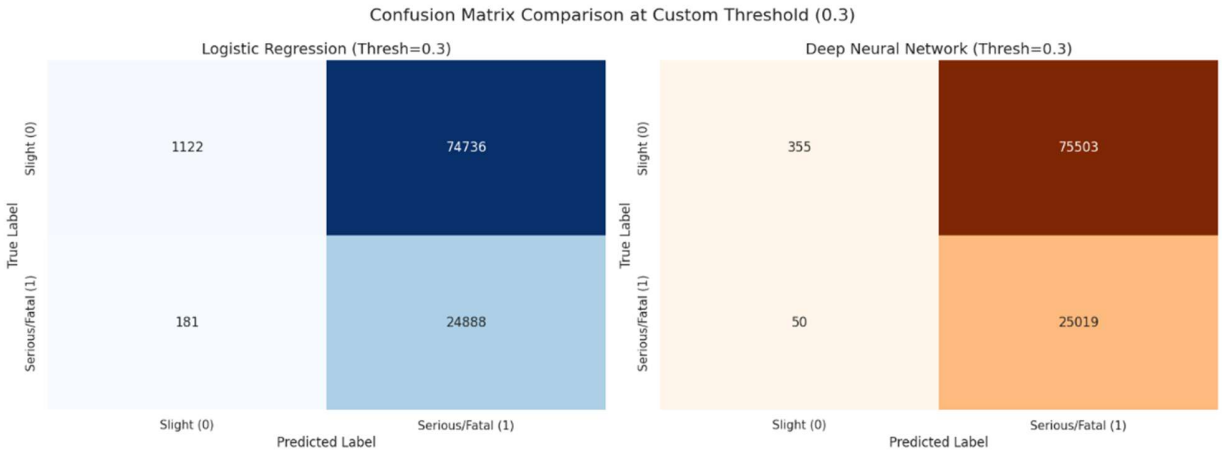


Figure 14. Confusion matrices at the default 0.3 threshold — Logistic Regression (left) vs DNN (right).

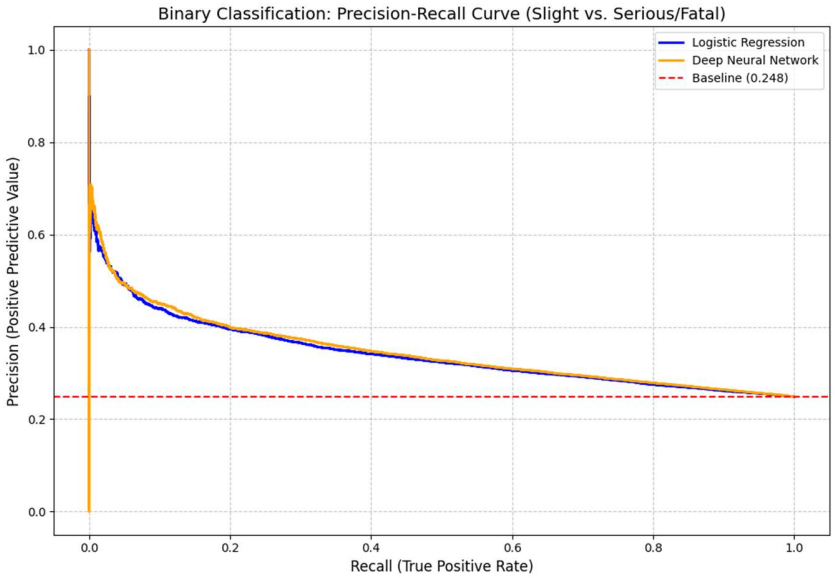


Figure 15. Precision–recall curve for the positive (Serious/Fatal) class.

4.3 Feature Engineering, Class Imbalance, and Comparison with the Three-class Problem

4.3.1 Pipeline and Imbalance Strategy:

I used the same preprocessing pipeline developed in Task 1.1 on the binary models. To handle the 75/25 binary split, imbalance strategies were mathematically updated:

- The Logistic Regression pipeline retained the class_weight='balanced' parameter

- The DNN was structurally modified to output a single node using a sigmoid activation function and compiled using binary_crossentropy loss. I applied a new custom binary class-weight dictionary to the training loop, weighting the majority class at ~0.66 and heavily penalising missed severe crashes by weighting the minority class at ~2.01. Overfitting was again controlled via Dropout(0.3) layers and EarlyStopping.

4.3.2 Comparison: Is the Binary Task Easier?

Reformulating the problem as a binary classification (Slight vs. KSI) conceptually simplifies the task for the algorithms, leading to more robust overall metrics.

In the three-class formulation (Task 1.2), the models struggled significantly to distinguish between 'Serious' and 'Fatal' collisions. Environmentally, these two classes are nearly identical, they both tend to occur at high speeds, on rural roads, or at night. The difference between a Serious injury and a Fatal injury often comes down to micro-variables not present in this dataset.

By merging Serious and Fatal into a single KSI class, we eliminate the need for the model to learn an impossible boundary. The algorithm only has to differentiate between 'low energy/minor' collisions and 'high energy/severe' collisions. Consequently, the binary models demonstrated highly stable decision boundaries. The binary DNN achieved a Balanced Accuracy of 0.5796, a marked improvement over the 0.4763 achieved by the multi-class DNN.

5. Conclusion

Ultimately, the severe class imbalance of the DfT collision dataset dictated the entire methodological approach of this analysis. While the multi-class Keras Deep Neural Network (Task 1.2) established a strong mathematical baseline, its struggle to differentiate between environmentally identical 'Serious' and 'Fatal' classes proved that standard accuracy metrics are insufficient for life-or-death dispatch protocols. By reformulating the problem into a Binary Classification task (Task 1.3), the predictive signal was successfully clarified. When paired with an empirically lowered decision threshold, the Binary DNN emerges as the optimal model for this application. It successfully prioritises the critical minority class (KSI collisions), effectively minimising fatal False Negatives at the acceptable, real-world cost of manageable False Positives.